Práctica 2. Código seguro: del dato al fallo

Nombre y apellidos: _____

Hasta ahora hemos estado con Git y Docker. A partir de hoy los usamos como herramientas de trabajo y nos metemos en lo que de verdad importa en este módulo: leer código buscando por dónde puede entrar un ataque, y demostrar con pruebas que un fallo está arreglado y no va a volver. La práctica es **individual**.

Antes de empezar, un aviso: `validar.py` ejecuta órdenes de verdad en tu ordenador. Si pruebas cosas raras, que sean inofensivas (`id`, `whoami`). Nada de borrar ficheros.

1. Prepara el entorno

Descarga `PPS-P2-codigo-seguro-LAB.zip` del Drive (carpeta 01 ALUMNADO) y descomprímelo dentro de tu carpeta `pps-arranque`. Después crea un entorno virtual e instala pytest y coverage:

```
cd pps-arranque
unzip ~/Descargas/PPS-P2-codigo-seguro-LAB.zip -d .
python3 -m venv .venv
. .venv/bin/activate
pip install pytest coverage
```

Asegúrate de que la carpeta `.venv` no se va a subir al repositorio:

```
grep -qx '.venv/' .gitignore || echo '.venv/' >> .gitignore
```

Si algo falla en este paso, avísame en cuanto te pase y sigue con el apartado 2.

2. Del dato de entrada al sink (PPS-E1)

Entra en la carpeta del validador y ejecuta estas dos órdenes:

```
cd ut1/validador
python3 validar.py ana
python3 validar.py "' OR '1'='1"
```

Fíjate en qué pasa con la segunda y en qué línea se para el programa. ¿Qué crees que significa?

Ahora lee los tres ficheros (`validar.py`, `Validar.java` y `validar.c`). En cada uno tienes que encontrar por dónde entra el dato (la **entrada**), dónde puede hacer daño (el **sink**), si hay alguna validación y si sirve para algo, y por qué falla teniendo en cuenta cómo se ejecuta ese lenguaje: interpretado, máquina virtual o compilado.

Copia la plantilla y rellénala. Al final pon también cómo lo arreglarías:

```
cd ..
cp PPS-E1-lectura-PLANTILLA.md PPS-E1-lectura.md
```

3. Los tests pasan, pero… (PPS-E2)

Entra en el miniproyecto y pasa los tests midiendo la cobertura:

```
cd miniproyecto
coverage run -m pytest -q
coverage report -m
```

Lee la incidencia #17 que hay al principio de `descuentos.py`. Apunta la cobertura que te sale. ¿Hay algún test que se dé cuenta del fallo?

4. Test de regresión (PPS-E2)

Al final de `test_descuentos.py` escribe una función `test_regresion_incidencia_17` que compruebe que un descuento del 150 % y otro del -5 % lanzan `ValueError` (usa `pytest.raises`).

Pasa los tests. **El tuyo tiene que fallar.** Si te sale bien a la primera, es que no está comprobando el fallo. Cuando lo veas fallar, arregla `aplicar_descuento` en `descuentos.py` y vuelve a pasarlos hasta que estén todos bien.

5. Fuzzing

Con la función ya arreglada, ejecuta:

```
python3 fuzz_casero.py
```

Este programa prueba 20.000 combinaciones de valores raros contra tu función y avisa cuando algo no cuadra. Mira qué encuentra. Después:

1. Completa la función `resultado_valido()` (donde pone TODO). Piensa qué tiene que cumplir siempre un precio después de aplicarle un descuento. Te puede venir bien el módulo `math`.
2. Arregla `aplicar_descuento` para que rechace esos valores.
3. Añade un test de regresión para lo que has encontrado.

Has terminado cuando el fuzzing dé 0 fallos y todos los tests pasen.

Si te atascas

- Si te sale `externally-managed-environment`, estás usando el pip del sistema. Activa el entorno virtual con `. .venv/bin/activate`.
- Si te sale `No module named pytest`, seguramente has abierto otro terminal y no has activado el entorno virtual.
- Si te sale `ModuleNotFoundError: No module named 'descuentos'`, no estás dentro de `ut1/miniproyecto`.

6. Entrega

Contesta las preguntas de abajo y sube la carpeta `ut1` a tu repositorio. Antes de hacer el commit, comprueba con `git status` que no aparece `.venv`.

```
git add ut1
git commit -m "Práctica 2: PPS-E1 y PPS-E2"
git push
```

Preguntas

1. ¿Cuál es el sink más peligroso de `validar.py`? ¿Por qué?
2. ¿Qué cobertura tenía `descuentos.py` con el fallo dentro?
3. ¿Qué mensaje te dio pytest cuando tu test fallaba?
4. ¿Qué valores encontró el fuzzing que tus tests no habían probado?
5. URL de tu repositorio.